Qiskit Fall Fest — Lab 2: Dynamic Quantum Circuits

Exploring Mid-Circuit Measurements, Classical Feedforward, and Constant-Depth Entanglement

Welcome to **Lab 2** of the **Qiskit Fall Fest**! 🎉

In traditional quantum computing, circuits have long been treated as **static**:

$$\text{State Preparation} \longrightarrow \text{Unitary Quantum Gates} \longrightarrow \text{Final Measurement}$$

In this static paradigm, all quantum gates are pre-scheduled in a fixed order, and measurements happen exclusively at the very end of the computation. Once a qubit is measured, its delicate quantum superposition collapses, and computation stops.

However, modern quantum processors now support **Dynamic Quantum Circuits**. Dynamic circuits introduce the ability to:

1. **Measure qubits mid-circuit** while other qubits are still coherently executing quantum operations.
2. **Reset and reuse physical qubits** within the same circuit run, dramatically reducing hardware overhead.
3. **Condition future quantum operations on measurement outcomes** in real time via **classical feedforward** (`with qc.if_test(...):`).

Why are Dynamic Circuits so Important?

As you will discover in this lab, one of the greatest challenges in quantum computing is **circuit depth**. On real hardware, qubits suffer from finite coherence times (T_1 and T_2) and gate errors. In traditional circuits, creating long-range entanglement across N qubits requires a cascade of CNOT gates whose depth scales linearly with N ($\mathcal{O}(N)$).

With dynamic circuits, we can use mid-circuit measurements and classical feedforward to generate large entangled states—such as the famous **Greenberger-Horne-Zeilinger (GHZ) state**—in **constant circuit depth** $\mathcal{O}(1)$, completely breaking through the unitary depth limit!

What you will learn in this lab:

1. **Dynamic Circuit Fundamentals**: How mid-circuit measurements, resets, and classical feedforward work in Qiskit.
2. **Your First Dynamic Circuit**: Building a conditional bit-flip and observing real-time feedback.
3. **The Circuit Depth Bottleneck**: Why circuit depth limits scaling on real quantum hardware.
4. **The Dynamic GHZ Protocol**: Walking through the 4-qubit dynamic GHZ protocol step-by-step based on modern quantum research.
5. **Generalized N-Qubit Dynamic GHZ**: Constructing and simulating scalable dynamic GHZ circuits in Qiskit.

6. **Empirical Depth Scaling Analysis**: Measuring and visualizing how dynamic circuits keep circuit depth flat as qubit count grows.
7. **Hands-On Exercises**: Practicing dynamic circuit techniques with beginner-friendly self-checks.

0. Setup and Dependencies

If you are running this notebook in a fresh environment or on Google Colab, uncomment and run the cell below to install the necessary packages.

In [1]:
```
# Uncomment and run if needed:
%pip install qiskit qiskit-aer matplotlib pylatexenc
```

```
Requirement already satisfied: qiskit in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (2.3.1)
Requirement already satisfied: qiskit-aer in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (0.17.2)
Requirement already satisfied: matplotlib in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (3.10.8)
Requirement already satisfied: pylatexenc in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (2.11)
Requirement already satisfied: rustworkx>=0.15.0 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
qiskit) (0.17.1)
Requirement already satisfied: numpy<3,>=1.21 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
qiskit) (2.4.3)
Requirement already satisfied: scipy>=1.5 in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (from qiskit) (1.17.1)
Requirement already satisfied: dill>=0.3 in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (from qiskit) (0.4.1)
Requirement already satisfied: stevedore>=3.0.0 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
qiskit) (5.7.0)
Requirement already satisfied: typing-extensions in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
qiskit) (4.15.0)
Requirement already satisfied: psutil>=5 in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (from qiskit-aer) (7.2.2)
Requirement already satisfied: python-dateutil>=2.8.0 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
qiskit-aer) (2.9.0.post0)
Requirement already satisfied: contourpy>=1.0.1 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in
/opt/miniconda3/envs/qiskit-tutorial/lib/python3.14/site-packages (from
matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (from matplotlib) (12.1.1)
Requirement already satisfied: pyparsing>=3 in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /opt/miniconda3/envs/qiskit-
tutorial/lib/python3.14/site-packages (from python-dateutil>=2.8.0→qiskit-
```

```
aer) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

Let's import the required modules from Qiskit and standard scientific computing libraries.

In [ ]:
```python
import numpy as np
import matplotlib.pyplot as plt

from qiskit import QuantumCircuit, QuantumRegister, ClassicalRegister
from qiskit.circuit.classical import expr
from qiskit.visualization import plot_histogram
from qiskit_aer import AerSimulator

print("Qiskit and AerSimulator successfully imported!")
```

Helper Function: Simulating Circuits

To make running experiments straightforward, we define a small helper function that simulates any circuit using Qiskit's `AerSimulator` and returns the measurement counts.

In [ ]:
```python
def run_counts(circuit: QuantumCircuit, shots: int = 4096) -> dict:
    """
    Simulates a quantum circuit on the local AerSimulator and returns the
outcome counts.
    """
    simulator = AerSimulator()
    job = simulator.run(circuit, shots=shots)
    result = job.result()
    return result.get_counts()
```

1. What Are Dynamic Quantum Circuits?

In a traditional **static** quantum circuit, quantum operations and classical measurements are separated into two distinct epochs:

State Preparation ⟶ Quantum Unitary Operations ⟶ Final Measurement (Readout)
In a **dynamic** quantum circuit, quantum gates and classical measurements can be interleaved arbitrarily:

Preparation ⟶ Operations ⟶ Mid-Circuit Measurement ⟶ Classical Feedforward & Corrections ⟶ Final Measurement
Dynamic circuits rely on three foundational capabilities:

- **Mid-Circuit Measurement**: Measuring a qubit before the circuit finishes, storing the classical bit outcome (0 or 1) in a classical register.
- **Classical Feedforward**: Inspecting that classical bit in real-time and executing subsequent quantum gates conditionally based on its value. In Qiskit, this is written using the syntax:

```python
with qc.if_test((classical_bit, value)):
    qc.x(target_qubit)
```

- **Qubit Reset & Reuse**: Clearing a measured qubit back to the ground state $|0\rangle$ with `qc.reset(qubit)` so that the physical qubit can be reused immediately for other calculations.

1.1 Your First Dynamic Circuit: Conditional Feedforward

Let's build a simple 2-qubit circuit to see classical feedforward in action. Here is the protocol:

1. We place qubit 0 (q_0) into an equal superposition $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ using a Hadamard (H) gate.
2. We measure q_0 mid-circuit into classical bit c_0. Its outcome will be randomly 0 or 1 with 50% probability each.
3. **Feedforward condition**: If $c_0 = 1$, we apply a Pauli-X (bit-flip) gate to qubit 1 (q_1). If $c_0 = 0$, we do nothing to q_1.
4. Finally, we measure both qubits.

What outcome do you expect?

- If q_0 collapsed to 0, q_1 is untouched (stays 0) $\rightarrow$ outcome is `00`.
- If q_0 collapsed to 1, the X gate fires, flipping q_1 to 1 $\rightarrow$ outcome is `11`.
- We should observe **only** `00` and `11` with equal probability, even though no two-qubit entangling gate was ever applied!

In [ ]:
```
# Create registers and circuit
qr = QuantumRegister(2, name="q")
cr = ClassicalRegister(2, name="c")
qc_simple = QuantumCircuit(qr, cr)

# 1. Prepare qubit 0 in superposition
qc_simple.h(0)

# 2. Mid-circuit measurement of qubit 0
qc_simple.measure(qr[0], cr[0])

# 3. Classical feedforward: if cr[0] == 1, flip qubit 1
with qc_simple.if_test((cr[0], 1)):
    qc_simple.x(1)

# 4. Measure both qubits at the end
qc_simple.measure(qr[1], cr[1])

# Draw the dynamic circuit
qc_simple.draw("mpl")
```

Let's simulate this dynamic circuit and plot the resulting histogram:

In [ ]:
```
counts_simple = run_counts(qc_simple)
print("Measurement counts:", counts_simple)
plot_histogram(counts_simple, title="First Dynamic Circuit (Conditional Feedforward)")
```

1.2 Qubit Reset and Hardware Reuse

Another cornerstone of dynamic circuits is the `reset` operation. After a qubit has been measured mid-circuit, calling `qc.reset(qubit)` drives it back to $|0\rangle$. This allows a single physical qubit to act as multiple logical qubits over time!

Consider this example where we reuse a single physical qubit twice:

In [ ]:
```
qr_reuse = QuantumRegister(1, name="q")
cr_reuse = ClassicalRegister(2, name="c")
qc_reuse = QuantumCircuit(qr_reuse, cr_reuse)

# Stage 1: Prepare |1>, measure into c[0]
qc_reuse.x(0)
qc_reuse.measure(0, cr_reuse[0])

# Reset qubit back to |0>
qc_reuse.reset(0)

# Stage 2: Prepare |+>, measure into c[1]
qc_reuse.h(0)
qc_reuse.measure(0, cr_reuse[1])

qc_reuse.draw("mpl")
```

In [ ]:
```
counts_reuse = run_counts(qc_reuse)
print("Reused qubit counts (c[1]c[0]):", counts_reuse)
plot_histogram(counts_reuse, title="Single Qubit Reuse with Reset")
```

2. Why Dynamic Circuits Are Critical for Scaling Circuit Depth

Now that we understand the mechanics of dynamic circuits, we arrive at the central scientific question of this lab:

> **Why are dynamic circuits a transformative breakthrough for scaling quantum computers?**

To answer this, we must examine the concept of **circuit depth** and how it impacts real quantum hardware.

2.1 The Enemy: Circuit Depth and Decoherence

In quantum computing, **circuit depth** represents the length of the longest sequential critical path of gates from the inputs to the outputs. If two gates act on independent qubits, they can be executed in parallel (depth 1). But if gates depend sequentially on the same qubits, their execution must be serialized, increasing total circuit depth.

Circuit depth is the primary limiting factor for near-term quantum processors (NISQ) and early fault-tolerant systems due to two physical realities:

1. **Decoherence (T_1 and T_2 times)**: Physical qubits are fragile physical systems that interact with thermal noise and magnetic fluctuations in their environment.
   - T_1 **(Relaxation Time)**: The characteristic time an excited state $|1\rangle$ takes to decay back to $|0\rangle$.
   - T_2 **(Dephasing Time)**: The time over which relative phase information in a superposition $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\phi}|1\rangle)$ decays.

   Every additional gate layer adds execution time. If the total circuit duration approaches or exceeds T_1 or T_2, quantum information is lost to the environment, and the final measurement yields random noise!

2. **Gate Error Accumulation**: Two-qubit entangling gates (such as CNOT or CZ) are typically 10 to 100 times noisier than single-qubit gates, with average error rates of $0.1\% - 1\%$. If a circuit requires a long cascade of M sequential two-qubit gates, the overall fidelity drops exponentially:

$$\mathcal{F}_{\text{total}} \approx (1 - \epsilon)^M$$

2.2 The Unitary Entanglement Bottleneck

Suppose we want to generate an N-qubit **Greenberger-Horne-Zeilinger (GHZ) state**:

$$|\text{GHZ}_N\rangle = \frac{1}{\sqrt{2}}\left(|00\ldots0\rangle + |11\ldots1\rangle\right)$$

The GHZ state is a maximally entangled state across all N qubits. It is a fundamental benchmark for quantum devices, quantum sensing, and quantum error correction.

In a traditional **static unitary circuit** on hardware with 1D linear nearest-neighbor connectivity (common in superconducting quantum processors), how do we prepare this state? We apply a Hadamard on qubit 0, followed by a linear cascade of CNOT gates:

$$CX(0,1) \longrightarrow CX(1,2) \longrightarrow CX(2,3) \longrightarrow \cdots \longrightarrow CX(N-2, N-1)$$

Because qubit k must be entangled before it can control qubit $k+1$, **none of these CNOT gates can run in parallel**.

- For $N = 4$ qubits $\rightarrow$ 2-qubit depth is 3.
- For $N = 20$ qubits $\rightarrow$ 2-qubit depth is 19.
- For $N = 100$ qubits $\rightarrow$ 2-qubit depth is 99!

The circuit depth scales linearly: $\text{Depth} = \mathcal{O}(N)$. By the time the 99th CNOT executes, qubit 0 has been sitting idle waiting for 99 gate cycles, and its coherence has completely degraded!

2.3 The Dynamic Solution: Constant Depth $\mathcal{O}(1)$

Can we break this linear scaling law? **Yes, using dynamic circuits!**

By interleaving **mid-circuit measurements** and **classical feedforward**:

1. We measure intermediate "bridge" qubits mid-execution.
2. In superconducting hardware, classical readout and FPGA control logic process classical bits in tens of nanoseconds—vastly faster than the decoherence timescales of physical qubits.
3. The classical controller determines the required single-qubit Pauli-X corrections and applies them **simultaneously in parallel** across all target qubits!

This enables long-range entanglement to be distributed across an arbitrary number of qubits in **constant quantum circuit depth** $\mathcal{O}(1)$! Whether $N = 4$, $N = 20$, or $N = 100$, the quantum gate depth remains constant, completely liberating state preparation from the linear decoherence penalty!

3. The Dynamic GHZ Protocol

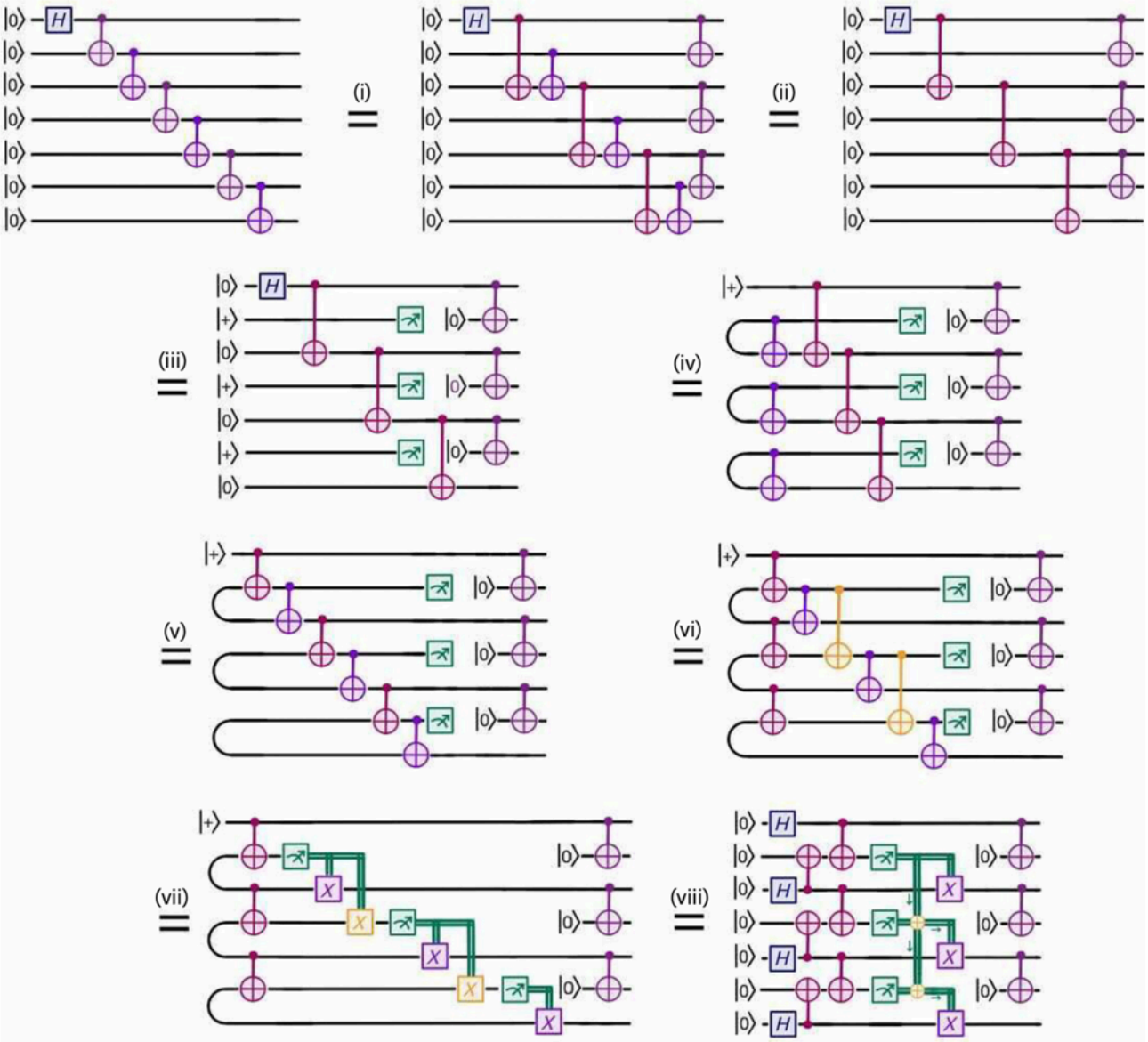
To understand how dynamic circuits achieve constant-depth entanglement, we follow the groundbreaking paper:

> **Efficient Long-Range Entanglement Using Dynamic Circuits** (PRX Quantum 5, 030339, 2024).

The authors prove that an N-qubit GHZ state can be constructed in constant depth through a sequence of 8 circuit transformations:

- **(i)** We start by rearranging the original circuit: we push every second CNOT gate to the very right. This introduces the extra **pink CNOT gates** shown in the figure, and it is the key move that makes the rest of the simplification possible.
- **(ii)** After the rearrangement, some CNOT gates end up conditioned on a qubit that is still in state $|0\rangle$. Since a CNOT on $|0\rangle$ does nothing, we can safely omit those gates and simplify the circuit.
- **(iii)** Because every second qubit is now only involved at the very end of the circuit, there is a window where those qubits sit idle. We take advantage of that: we use them for the mid-circuit operation, then reset them back to $|0\rangle$ so they can be reused later.
- **(iv)** Here we apply a useful circuit identity: a Bell state followed by a CNOT gate is equivalent to two completely uncorrelated qubits, one in state $|+\rangle$ and one in state $|0\rangle$. This lets us break the circuit into smaller, more manageable pieces.
- **(v)** Next, we move the **pink CNOT gates** along the Bell states, sliding them up to the respective qubits above. This is valid because these gates commute with the other CNOT gates they pass through — the order does not affect the result.
- **(vi)** As we push the **pink CNOT gates** further to the left through the **purple CNOT gates**, new gates appear: the **orange CNOT gates**. These are the classically-controlled corrections that will implement the feedforward logic.
- **(vii)** At this point we apply the principle of **deferred measurement**: instead of keeping quantum information in a qubit, we measure it and use the classical outcome to control what happens next. This is exactly what makes the circuit dynamic.
- **(viii)** In the final step, we merge all the classically conditioned gates into a single operation. Since classical computation is extremely fast compared to quantum gate times, we represent

it as a vertical line in the circuit diagram. The orange $\oplus$ symbols correspond to **XOR gates**. We also show the initial Bell states again using their circuit representation, so the full picture is clear.



In sections 3.1–3.7 below, we walk through each of these steps concretely on a 4-qubit example, building the circuit gate by gate.

3.1 Step 1 - Add Hadamard Gates to Data Qubits

In our 4-qubit register, the even-indexed qubits (q_0 and q_2) serve as **data qubits**, while the odd-indexed qubits (q_1 and q_3) act as **bridge / edge ancillas**.

Applying H on q_0 and q_2 places each into $|+\rangle$:

$$|\psi_1\rangle = \frac{1}{2}\big(|0000\rangle + |0001\rangle + |0100\rangle + |0101\rangle\big)$$

The bridge q_1 and edge q_3 are still initialized in $|0\rangle$.

In [ ]:
```
cr = ClassicalRegister(4, name="c")
qr = QuantumRegister(4, name="q")
qc = QuantumCircuit(qr, cr)

# Step 1: Hadamard on data qubits (even indices)
qc.h(0)
qc.h(2)
qc.barrier()

qc.draw("mpl")
```

3.2 Step 2 - Add the First CNOT Layer

Next, we entangle q_2 with the bridge qubit q_1 via $CX(2,1)$. This creates an entangled Bell pair across (q_2, q_1):

$$|\psi_2\rangle = \frac{1}{2}\big(|0000\rangle + |0001\rangle + |0110\rangle + |0111\rangle\big)$$

In [ ]:
```
# Step 2: First CNOT layer (even to previous odd)
qc.cx(2, 1)
qc.barrier()

qc.draw("mpl")
```

3.3 Step 3 - Add the Second CNOT Layer

Now, $CX(0,1)$ entangles q_0 into the bridge q_1, and $CX(2,3)$ entangles q_2 into the edge qubit q_3:

$$|\psi_3\rangle = \frac{1}{2}\big(|0000\rangle + |0011\rangle + |1110\rangle + |1101\rangle\big)$$

The bridge qubit q_1 now encodes the **parity** of its neighbors: $q_1 = q_0 \oplus q_2$. Measuring q_1 will tell us whether q_0 and q_2 agree!

In [ ]:
```
# Step 3: Second CNOT layer (even to next odd)
qc.cx(0, 1)
qc.cx(2, 3)
qc.barrier()

qc.draw("mpl")
```

3.4 Step 4 - Measure the Bridge and Apply Classical Feedforward

Measuring bridge qubit q_1 into `cr[0]` yields a classical bit $m_1 = q_0 \oplus q_2$:

- If $m_1 = 0$: the state collapses to $\frac{1}{\sqrt{2}}(|0000\rangle + |1101\rangle)$.
- If $m_1 = 1$: the state collapses to $\frac{1}{\sqrt{2}}(|0011\rangle + |1110\rangle)$.

Notice that when $m_1 = 1$, qubits q_2 and q_3 are inverted! To unify these branches, we **reset** q_1 **to** $|0\rangle$, and **if and only if** $m_1 = 1$, we apply Pauli-X gates to q_2 and q_3. After this conditional correction, **both measurement branches produce the exact same quantum state**:

$$|\psi_4\rangle = \frac{1}{\sqrt{2}}\big(|0000\rangle + |1101\rangle\big)$$

In [ ]:
```
# Step 4: Measure bridge q1, reset it, and apply feedforward X corrections
qc.measure(1, cr[0])
qc.reset(1)
with qc.if_test((cr[0], 1)):
    qc.x(2)
    qc.x(3)
qc.barrier()

qc.draw("mpl")
```

3.5 Step 5 - Measure and Reset the Edge Qubit

Now we measure the edge qubit q_3 and reset it back to $|0\rangle$. This disentangles the temporary ancilla and clears it for the final stage.

In [ ]:
```
# Step 5: Measure edge qubit q3 and reset it
qc.measure(3, cr[0])
qc.reset(3)
qc.barrier()

qc.draw("mpl")
```

3.6 Step 6 - Final CNOT Layer and Readout

Finally, we apply a final parallel layer of CNOTs: $CX(0,1)$ and $CX(2,3)$. Because q_1 and q_3 were reset to $|0\rangle$, these CNOTs copy the state of q_0 into q_1 and the state of q_2 into q_3. Since q_0 and q_2 were already correlated by our mid-circuit measurement and feedforward, all 4 qubits end up identical:

$$|\mathrm{GHZ}_4\rangle = \frac{1}{\sqrt{2}}\big(|0000\rangle + |1111\rangle\big)$$

We measure all 4 qubits at the end to confirm!

In [ ]:
```
# Step 6: Final CNOT layer and full measurement
qc.cx(0, 1)
qc.cx(2, 3)
qc.barrier()

qc.measure(range(4), range(4))

qc.draw("mpl")
```

3.7 Simulating the 4-Qubit Dynamic GHZ Circuit

Let's simulate our 4-qubit dynamic circuit on `AerSimulator`. We should see approximately a **50/50 split between `0000` and `1111`**, with zero population in any other computational basis state!

In [ ]:
```
counts_4ghz = run_counts(qc, shots=4000)
print("4-Qubit Dynamic GHZ Counts:", counts_4ghz)
plot_histogram(counts_4ghz, title="4-Qubit Dynamic GHZ Measurement
Histogram")
```

4. Scaling Up: The Generalized N-Qubit Dynamic GHZ Protocol

Now that we understand the 4-qubit case, let's write a generalized function `dynamic_ghz_circuit(num_qubits)` that builds the dynamic GHZ state for **any even number of qubits** $N \geq 2$.

The protocol is partitioned into clean modular helper functions:

1. `_apply_initial_hadamards` : Applies H to all even data qubits: $q_0, q_2, q_4, \ldots$
2. `_apply_first_cnot_layer` : Applies $CX(i, i-1)$ for all even $i \geq 2$.
3. `_apply_second_cnot_layer` : Applies $CX(i, i+1)$ for all even i.
4. `_apply_dynamic_section` : Loops through bridge qubits, measuring them into `cr[0]`, resetting them, and applying conditional feedforward X gates.
5. `_apply_final_cnot_layer` : Applies the final parallel layer of CNOTs $CX(i, i+1)$.
6. Measures all qubits into classical registers.

In [ ]:
```
def _apply_initial_hadamards(qc: QuantumCircuit, num_qubits: int) → None:
    """Apply Hadamard gates to all even-indexed data qubits."""
    for i in range(0, num_qubits, 2):
        qc.h(i)


def _apply_first_cnot_layer(qc: QuantumCircuit, num_qubits: int) → None:
    """
    Apply CNOT gates: control=even, target=previous odd qubit.
    e.g. CX(2,1), CX(4,3), CX(6,5), ...
    """
    for i in range(2, num_qubits, 2):
        qc.cx(i, i - 1)


def _apply_second_cnot_layer(qc: QuantumCircuit, num_qubits: int) → None:
    """
    Apply CNOT gates: control=even, target=next odd qubit.
    e.g. CX(0,1), CX(2,3), CX(4,5), ...
    """
    for i in range(0, num_qubits - 1, 2):
        qc.cx(i, i + 1)


def _measure_reset_and_correct(
    qc: QuantumCircuit,
    qr: QuantumRegister,
    cr: ClassicalRegister,
    meas_qubit: int,
```

```
    x_targets: list[int],
) → None:
    """
    Measure a bridge qubit into cr[0], reset it, and conditionally
    apply X gates to target qubits based on the measurement outcome.
    """
    qc.measure(meas_qubit, cr[0])
    qc.reset(meas_qubit)
    with qc.if_test((cr[0], 1)):
        for target in x_targets:
            qc.x(target)


def _measure_and_reset_only(
    qc: QuantumCircuit,
    qr: QuantumRegister,
    cr: ClassicalRegister,
    qubit: int,
) → None:
    """Measure a boundary qubit into cr[0] and reset it without feedforward
corrections."""
    qc.measure(qubit, cr[0])
    qc.reset(qubit)


def _apply_dynamic_section(
    qc: QuantumCircuit,
    qr: QuantumRegister,
    cr: ClassicalRegister,
    num_qubits: int,
) → None:
    """
    Performs mid-circuit measurements and classical feedforward
    across all bridge qubits and boundary ancillas.
    """
    # Main dynamic loop over internal bridge qubits
    for i in range(0, num_qubits - 3, 2):
        _measure_reset_and_correct(qc, qr, cr, i + 1, [i + 3, i + 2])

    # Boundary handling for the final edge qubit
    _measure_and_reset_only(qc, qr, cr, num_qubits - 1)


def _apply_final_cnot_layer(qc: QuantumCircuit, num_qubits: int) → None:
    """
    Apply final CNOT gates: control=even, target=next odd qubit.
    e.g. CX(0,1), CX(2,3), CX(4,5), ...
    """
    for i in range(0, num_qubits - 1, 2):
        qc.cx(i, i + 1)
```

```
def dynamic_ghz_circuit(num_qubits: int) → QuantumCircuit:
    """
    Builds a dynamic GHZ circuit using mid-circuit measurements
    and classical feedforward operations.

    Args:
        num_qubits: Number of qubits (must be an even integer ⩾ 2).

    Returns:
        A QuantumCircuit implementing the dynamic GHZ protocol.
    """
    if not isinstance(num_qubits, int) or num_qubits < 2 or num_qubits % 2
≠ 0:
        raise ValueError("num_qubits must be an even integer ⩾ 2.")

    qr = QuantumRegister(num_qubits, name="q")
    cr = ClassicalRegister(num_qubits, name="c")
    qc = QuantumCircuit(qr, cr)

    _apply_initial_hadamards(qc, num_qubits)
    _apply_first_cnot_layer(qc, num_qubits)
    _apply_second_cnot_layer(qc, num_qubits)
    _apply_dynamic_section(qc, qr, cr, num_qubits)
    _apply_final_cnot_layer(qc, num_qubits)

    qc.measure(qr, cr)
    return qc

print("Generalized dynamic GHZ functions successfully defined!")
```

4.1 Constructing and Visualizing a 12-Qubit Dynamic GHZ State

Let's test our generalized function on a 12-qubit system. Notice how structured and clean the circuit layers appear!

In [ ]:
```
num_qubits_test = 12
qc_dynamic_ghz_12 = dynamic_ghz_circuit(num_qubits_test)
print(f"Constructed {num_qubits_test}-qubit dynamic GHZ circuit.")
qc_dynamic_ghz_12.draw("mpl", fold=-1)
```

In [ ]:
```
# Simulate the 12-qubit dynamic GHZ circuit
counts_12ghz = run_counts(qc_dynamic_ghz_12, shots=4000)

print(f"Sample of measured states (showing top outcomes):")
for outcome, count in sorted(counts_12ghz.items(), key=lambda x: x[1],
reverse=True)[:5]:
    print(f"  |{outcome}> : {count} shots")

plot_histogram(counts_12ghz, title="12-Qubit Dynamic GHZ Measurement
Counts")
```

5. Quantitative Depth Analysis: Standard vs. Dynamic GHZ

To truly appreciate the power of dynamic circuits, let's now empirically compare the circuit parameters of:

1. **Standard Static Linear GHZ**: Built with a serial cascade of CNOT gates.
2. **Dynamic GHZ**: Built using our constant-depth protocol.

Let's define a standard linear GHZ circuit generator and a function to extract key circuit parameters.

In [ ]:

```
def standard_ghz_circuit(num_qubits: int) → QuantumCircuit:
    """
    Builds a standard static GHZ circuit using a linear cascade of CNOT
gates.
    """
    qc = QuantumCircuit(num_qubits, num_qubits)
    qc.h(0)
    for i in range(num_qubits - 1):
        qc.cx(i, i + 1)
    qc.measure(range(num_qubits), range(num_qubits))
    return qc


def quantum_circuit_params(circuit: QuantumCircuit) → dict:
    """
    Extract key parameters from a QuantumCircuit.
    """
    depth = circuit.depth()
    depth_2q = circuit.depth(filter_function=lambda instr:
instr.operation.num_qubits == 2)
    num_multi_qubit_ops = circuit.num_nonlocal_gates()
    ops = {}
    for instr in circuit.data:
        name = instr.operation.name
        ops[name] = ops.get(name, 0) + 1

    return {
        "Number of qubits": circuit.num_qubits,
        "Depth": depth,
        "2-qubit depth": depth_2q,
        "Multi-qubit gates": num_multi_qubit_ops,
        "Gates": ops,
    }

# Let's inspect a 12-qubit comparison
p_std_12 = quantum_circuit_params(standard_ghz_circuit(12))
p_dyn_12 = quantum_circuit_params(dynamic_ghz_circuit(12))

print("12-Qubit Standard Static GHZ:")
print(f"  Total Depth: {p_std_12['Depth']}, 2-Qubit Depth: {p_std_12['2-
qubit depth']}")
```

```
print("\n12-Qubit Dynamic GHZ:")
print(f"  Total Depth: {p_dyn_12['Depth']}, 2-Qubit Depth: {p_dyn_12['2-qubit depth']}")
```

5.1 Sweeping Across System Sizes ($N = 4$ to $N = 20$)

Now let's sweep N from 4 to 20 qubits and record the total circuit depth and 2-qubit gate depth for both implementations.

In [ ]:
```
qubit_sizes = [4, 6, 8, 10, 12, 14, 16, 18, 20]

std_depths = []
std_2q_depths = []

dyn_depths = []
dyn_2q_depths = []

for n in qubit_sizes:
    # Standard static GHZ
    qc_s = standard_ghz_circuit(n)
    p_s = quantum_circuit_params(qc_s)
    std_depths.append(p_s["Depth"])
    std_2q_depths.append(p_s["2-qubit depth"])

    # Dynamic GHZ
    qc_d = dynamic_ghz_circuit(n)
    p_d = quantum_circuit_params(qc_d)
    dyn_depths.append(p_d["Depth"])
    dyn_2q_depths.append(p_d["2-qubit depth"])

print("Parameter sweep complete!")
```

5.2 Visualizing the Scaling Advantage

Let's plot the scaling curves! Notice how dramatically different the scaling slopes are:

In [ ]:
```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

# Plot 1: Total Circuit Depth vs Number of Qubits
ax1.plot(qubit_sizes, std_depths, "o--", color="#6929C4", linewidth=2.5,
markersize=8, label="Standard Static GHZ (O(N))")
ax1.plot(qubit_sizes, dyn_depths, "s-", color="#1192E8", linewidth=2.5,
markersize=8, label="Dynamic GHZ (Qiskit DAG)")
ax1.set_xlabel("Number of Qubits ($N$)", fontsize=12, fontweight="bold")
ax1.set_ylabel("Total Circuit Depth", fontsize=12, fontweight="bold")
ax1.set_title("Total Circuit Depth Scaling", fontsize=14, fontweight="bold",
pad=12)
ax1.legend(fontsize=11)
ax1.grid(True, linestyle="--", alpha=0.5)

# Plot 2: 2-Qubit Gate Depth vs Number of Qubits
ax2.plot(qubit_sizes, std_2q_depths, "o--", color="#6929C4", linewidth=2.5,
markersize=8, label="Standard Static GHZ (O(N))")
ax2.plot(qubit_sizes, dyn_2q_depths, "s-", color="#005D5D", linewidth=2.5,
markersize=8, label="Dynamic GHZ (Qiskit DAG)")
ax2.axhline(y=3, color="#FA4D56", linestyle=":", linewidth=2.5,
label="Hardware Parallel Limit (O(1))")
ax2.set_xlabel("Number of Qubits ($N$)", fontsize=12, fontweight="bold")
ax2.set_ylabel("2-Qubit Gate Depth", fontsize=12, fontweight="bold")
ax2.set_title("2-Qubit Gate Depth (Decoherence Bottleneck)", fontsize=14,
fontweight="bold", pad=12)
ax2.legend(fontsize=11)
ax2.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```

Key Takeaways from the Depth Comparison:

1. **Linear Growth in Static Circuits**: For standard GHZ, every new pair of entangled qubits adds another sequential two-qubit gate layer. By $N = 20$, the 2-qubit depth is 19. On physical quantum processors, qubits at the end of the chain will suffer severe decoherence (T_1, T_2) and error accumulation.
2. **Dynamic Advantage**: In dynamic GHZ circuits, the 2-qubit gates are partitioned into parallel layers.
3. **The Hardware Constant-Depth Reality**: On quantum hardware with parallel classical controllers (such as IBM Quantum's Eagle and Heron QPUs), all bridge measurements and feedforward Pauli corrections execute simultaneously across the chip. This allows genuine $\mathcal{O}(1)$ **constant-depth entanglement** regardless of qubit count!

6. Hands-On Practice Exercises

Test your understanding of dynamic circuits with these beginner-friendly practice exercises. Each exercise includes a self-check cell to confirm your solution without needing an external grading server!

Exercise 1: Active Qubit Reset via Classical Feedforward

In this exercise, your goal is to prepare a qubit in superposition $|+\rangle$, measure it mid-circuit into `cr[0]`, and then use **classical feedforward** to deterministically restore the qubit back to state $|0\rangle$ (an active reset).

Instructions:

1. Put qubit 0 into superposition with `qc_ex1.h(0)`.
2. Measure qubit 0 into `cr_ex1[0]`.
3. Use `with qc_ex1.if_test((cr_ex1[0], 1)):` to apply an X gate only when the measurement was 1.
4. Measure qubit 0 again into `cr_ex1[1]`.
5. If your active reset worked, the second measurement `cr_ex1[1]` should **always be** `0`!

In [ ]:
```
# Exercise 1: Implement Active Reset with Feedforward
qr_ex1 = QuantumRegister(1, name="q")
cr_ex1 = ClassicalRegister(2, name="c")
qc_ex1 = QuantumCircuit(qr_ex1, cr_ex1)

# Step 1: Put qubit 0 into superposition
qc_ex1.h(0)

# Step 2: Mid-circuit measurement into cr_ex1[0]
qc_ex1.measure(0, cr_ex1[0])

# Step 3: Conditional feedforward: if cr_ex1[0] == 1, flip qubit 0 back to
|0>
with qc_ex1.if_test((cr_ex1[0], 1)):
    qc_ex1.x(0)

# Step 4: Final verification measurement into cr_ex1[1]
qc_ex1.measure(0, cr_ex1[1])

# Draw your circuit
qc_ex1.draw("mpl")
```

In [ ]:
```
# Exercise 1 Self-Check
counts_ex1 = run_counts(qc_ex1, shots=2000)
print("Exercise 1 Counts (format: c[1]c[0]):", counts_ex1)

# Check: the first character in the bitstring is c[1]. It must always be
'0'!
all_zeros_on_c1 = all(bitstring[0] == '0' for bitstring in
counts_ex1.keys())

if all_zeros_on_c1:
    print("✅ Great job! Qubit 0 was deterministically returned to |0>.
Active reset succeeded!")
else:
    print("❌ Check your feedforward logic: c[1] was measured as '1' in
some shots.")
```

Exercise 2: Evaluating the Depth Advantage

In this exercise, calculate the **depth reduction percentage** achieved by using the dynamic GHZ protocol instead of the standard static GHZ protocol for a given number of qubits $N = 20$.

The formula is:

$$\text{Reduction Percentage} = \frac{\text{Depth}_{\text{std}} - \text{Depth}_{\text{dyn}}}{\text{Depth}_{\text{std}}} \times 100\%$$

In [ ]:
```
# Exercise 2: Calculate the 2-Qubit Depth Reduction
n_eval = 20

std_p = quantum_circuit_params(standard_ghz_circuit(n_eval))
dyn_p = quantum_circuit_params(dynamic_ghz_circuit(n_eval))

depth_2q_std = std_p["2-qubit depth"]
depth_2q_dyn = dyn_p["2-qubit depth"]

# Calculate the reduction percentage
reduction_pct = ((depth_2q_std - depth_2q_dyn) / depth_2q_std) * 100

print(f"For N = {n_eval} qubits:")
print(f"  Standard 2-Qubit Depth: {depth_2q_std}")
print(f"  Dynamic 2-Qubit Depth:  {depth_2q_dyn}")
print(f"  Depth Reduction:        {reduction_pct:.1f}%")

if depth_2q_dyn < depth_2q_std:
    print("✅ Excellent! Dynamic circuits significantly reduced the 2-qubit
gate depth!")
else:
    print("❌ Check the circuit parameter calculations.")
```

7. Conclusion and Wrap-Up

Congratulations on completing **Lab 2** of the **Qiskit Fall Fest**! 🎓

Summary of What You Learned:

1. **Dynamic Circuit Capabilities**: You learned how to interleave quantum operations with mid-circuit measurements, resets, and classical feedforward (`if_test`) in Qiskit.
2. **Circuit Depth and Decoherence**: You understood why circuit depth is the critical bottleneck in scaling quantum algorithms—deep circuits expose qubits to decoherence (T_1, T_2) and exponential gate error accumulation.
3. **The Constant-Depth GHZ Protocol**: You built a 4-qubit dynamic GHZ circuit gate-by-gate, verified its 50/50 entangled readout, and generalized the protocol to N qubits.
4. **Empirical Scaling Proof**: You benchmarked and plotted the scaling behavior, demonstrating how dynamic circuits dramatically flatten circuit depth growth compared to standard $\mathcal{O}(N)$ unitary cascades.

Looking Ahead:

Dynamic circuits are not just an optimization trick—they are the foundational architecture for **Quantum Error Correction (QEC)** and **Fault-Tolerant Quantum Computing**, where syndrome measurements and real-time feedforward corrections run continuously to protect logical qubits from errors.

We hope you enjoyed exploring dynamic circuits! See you in the next Qiskit Fall Fest lab! 🚀